

Taller Práctico Parte 1: Criptografía y Seguridad Básica con JWT

Refugio de Mascotas "Patitas al Rescate"

Arquitectura de Identidad del Refugio

Para dar soporte a nuestro refugio de mascotas, gestionaremos los accesos mediante dos perfiles claramente diferenciados:

1.  **ADMIN (Voluntario del Refugio)**: Encargado de registrar nuevos rescates, actualizar fichas de salud de los perritos y gatitos, y autorizar solicitudes de adopción.
 2.  **USER (Adoptante Registrado)**: Persona interesada en postularse para adoptar a una mascota, ver fotos de las mascotas disponibles y actualizar sus datos de contacto.
-

Hito 1: Registro y Login Vulnerable en Texto Plano

El objetivo de este hito es experimentar de manera directa el riesgo extremo de almacenar contraseñas legibles y por qué la comparación manual es una práctica inaceptable en producción.

1. Especificación del Modelo de Datos (PostgreSQL)

Para almacenar las credenciales de los voluntarios y adoptantes, utilizaremos la tabla `usuarios` en PostgreSQL

```
CREATE TABLE usuarios (  
  
    id SERIAL PRIMARY KEY,  
  
    username VARCHAR(50) NOT NULL UNIQUE,  
  
    password VARCHAR(100) NOT NULL,  
  
    rol VARCHAR(20) NOT NULL  
  
);
```

2. Entidad del Dominio en Java

Crea la clase de entidad `Usuario.java` en el paquete `com.krakedev.jwt.entidades`. Mapea de forma directa la tabla creada.

3. Capa de Datos (Repositorio)

Crea la interfaz `UsuarioRepository.java` en el paquete `com.krakedev.jwt.repositories` para realizar consultas a la base de datos:

4. Capa de Negocio (Servicio Vulnerable)

Crea la clase `UsuarioService.java` dentro de `com.krakedev.jwt.services`. En esta fase inicial, guardaremos la contraseña tal cual la recibimos en la base de datos, y realizaremos una validación por comparación directa `.equals()` en el login:

5. Controlador de Autenticación

Crea e inicializa la clase `AuthController.java` en el paquete `com.krakedev.jwt.controllers`. Expondremos los métodos HTTP para `/registrar` y `/login`:

WARNING

EJERCICIO DE LABORATORIO OBLIGATORIO (VULNERABILIDAD CRÍTICA):

1. Inicia tu servidor de Spring Boot.
2. Usando Postman, registra un nuevo adoptante realizando un `POST` a `http://localhost:8080/api/auth/registrar` con el cuerpo JSON:
3. json
4. {
5. `"username": "adoptante_pedro",`
6. `"password": "mi_clave_secreta_123",`
7. `"rol": "USER"`
8. }
9. Abre **pgAdmin**, ejecuta una consulta directa (`SELECT * FROM usuarios;`).
10. **Captura la pantalla** en donde se observe claramente el nombre de usuario y su contraseña en texto plano.
11. Escribe un análisis técnico (párrafo de 5 líneas) explicando cómo este esquema de almacenamiento permite que cualquier atacante interno (con acceso al servidor de BD) o externo (mediante SQL Injection) pueda comprometer todas las cuentas e identidades del refugio.
12. **Enviar La captura de Pantalla con el análisis al Grupo de WhatsApp**



Hito 2: Cifrado con BCrypt (Criptografía Unidireccional)

En este hito, erradicaremos la vulnerabilidad crítica del Hito 1 mediante la integración de **BCrypt**, un algoritmo robusto de hash diseñado especialmente para contraseñas.

1. Actualización de Dependencias

Abre tu archivo `pom.xml` y añade la librería `bcrypt` en tu bloque de dependencias:

xml

```
<dependency>
  <groupId>org.mindrot</groupId>
```

```
<artifactId>jbcrypt</artifactId>
<version>0.4</version>
</dependency>
```

2. Modificación del Servicio de Negocio

Actualizaremos UsuarioService.java para encriptar la contraseña mediante hashing al registrarse, y para validarla en el inicio de sesión a través del verificador criptográfico seguro:

EJERCICIO DE LABORATORIO Y REQUISITO DE ENTREGABLE:

1. Registra un nuevo usuario con rol de voluntario administrativo realizando un **POST** en Postman:
2. json
3. {
4. "username": "voluntario_carlos",
5. "password": "mi_clave_secreta_123",
6. "rol": "ADMIN"
7. }
8. Consulta en pgAdmin la tabla **usuarios** y **toma una captura** comparando visualmente el registro plano de **adoptante_pedro** con el nuevo registro de **voluntario_carlos** (cuyo hash debe empezar con **\$2a\$...**).
9. **[EVIDENCIA DE COMUNIDAD]**: Sube de inmediato esta captura de pantalla de pgAdmin al grupo de WhatsApp oficial del taller como evidencia de haber completado exitosamente la transición criptográfica.



Hito 3: Generación del Token JWT en el Login

Una vez protegida la persistencia, migraremos a una arquitectura **Stateless** (sin estado) para no depender de variables de sesión en memoria del servidor. Implementaremos **JSON Web Tokens (JWT)**.

1. Agregar Dependencia de JWT en Maven

Añade la librería oficial de Auth0 en tu archivo pom.xml:

```
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.4.0</version>
</dependency>
```

2. Diseñar la Utilidad de Tokens

Crea la clase JwtUtil.java en un nuevo paquete **com.krakedev.jwt.utils**. Esta clase encapsulará la firma, generación y descryptación de los tokens del refugio con una validez estricta de **30 minutos**:

3. Retornar el Token al Iniciar Sesión exitosamente

Modifica el método de login en tu controlador AuthController.java para que el servidor emita el token JWT con la estructura exacta de retorno JSON requerida:



Hito 4: Protección Manual del Perfil

El último hito de esta primera fase consiste en proteger un endpoint de forma directa en el controlador. Aprenderás a interceptar y desenscriptar la cabecera `Authorization` de forma manual.

1. Crear el Endpoint Protegido `/perfil`

Agrega el siguiente método GET al controlador AuthController.java:

2. Proceso de Prueba del Hito en Postman

Para validar tu implementación, realiza la siguiente secuencia de pruebas:

1. Realiza una petición POST a `/login` con las credenciales de `voluntario_carlos` y copia el token devuelto. **(Enviar al Grupo de WhatsApp el token devuelto desde postman)**
2. Crea una petición GET a `/perfil` en una nueva pestaña.
3. Dirígete a la pestaña **Headers** de la petición e ingresa manualmente la cabecera `Authorization` con el valor `Bearer <pega_tu_token_aquí>`.
4. Envía la petición y comprueba que se muestre el saludo dinámico específico para el rol del Voluntario.
5. Modifica de forma malintencionada un solo carácter del token JWT y ejecuta de nuevo la petición; comprueba cómo el servidor rechaza automáticamente el acceso devolviendo un código de estado `401 Unauthorized` debido a la rotura criptográfica de la firma digital.



Rúbrica y Criterios de Evaluación Cuantitativos

Para certificar que has completado satisfactoriamente los Hitos de la Parte 1 del Taller de Seguridad, deberás cumplir de manera exacta con la siguiente ponderación técnica de entregables:

1. Capturas enviadas al grupo de WhatsApp
2. Repositorio de GitHub